

# **PROJECT REPORT**

## **AI RESUME SCREENING STUDIO**

*An Intelligent Resume Parsing, Skill Extraction and Job Matching System*

### **CERTIFICATE**

This is to certify that the project entitled “AI Resume Screening Studio – An Intelligent Resume Parsing, Skill Extraction and Job Matching System” is a bonafide work carried out by the student named above under the guidance.

The work presented in this report is submitted in partial fulfillment of the requirements for the undergraduate programme.

### **DECLARATION**

I hereby declare that this project report titled “AI Resume Screening Studio” is my original academic work prepared for the purpose of partial fulfillment of the course requirements. The project has been developed as an academic software application for automated resume analysis and job-candidate matching.

# TABLE OF CONTENTS

## Abstract

1. Introduction
2. Problem Statement
3. Motivation
4. Objectives
5. Scope of the Project
6. Existing System
7. Proposed System
8. System Requirements
9. Technologies and Libraries Used
10. System Architecture
11. Input and Dataset Description
12. Resume Text Extraction and Preprocessing
13. Skill Extraction
14. Job Description Matching
15. Candidate Scoring and Ranking
16. Screening Algorithm
17. Detailed Module Description
18. User Interface and Interaction
19. Installation and Execution
20. Testing Strategy
21. Results and Discussion
22. Advantages
23. Limitations
24. Applications
25. Future Enhancements
26. Conclusion
27. References
28. Project File Structure

## Abstract

AI Resume Screening Studio is an intelligent resume-analysis application designed to assist in screening resumes against job requirements. The system accepts resume documents such as PDF and DOCX files, extracts their textual content, identifies relevant information such as skills, education, experience and certifications, and compares the extracted profile with a given job description.

The application is designed to reduce the manual effort involved in reviewing multiple resumes. Instead of depending only on a human reviewer to locate important information, the system organizes resume content into structured data and calculates a matching score based on the relationship between candidate skills and job requirements.

The project uses Python-based backend technologies and document-processing libraries. FastAPI provides the application backend, while PyPDF and python-docx are used for reading PDF and DOCX resumes. The system can be extended with Natural Language Processing and machine-learning techniques for improved skill recognition, semantic matching and candidate ranking.

The main purpose of the project is to demonstrate how Artificial Intelligence, text processing and automated screening can be combined into a practical recruitment-support application. The system is intended as an assistive screening tool and not as a replacement for final human recruitment decisions.

## 1. Introduction

Recruitment teams often receive a large number of resumes for a single job position. Manually reading each resume and comparing it with the job requirements can consume significant time. Important skills may also be missed when resumes use different formats, terminology or wording.

AI Resume Screening Studio addresses this problem by converting resume documents into machine-readable text and extracting important candidate information. The extracted information can then be compared with the requirements of a job description.

The project demonstrates practical use of Python, web application development, document parsing and text-processing techniques. It provides a foundation for building more advanced recruitment systems with semantic embeddings, machine learning and AI-assisted ranking.

## 2. Problem Statement

The problem addressed by this project is the difficulty of efficiently screening and comparing a large number of resumes against job requirements. Traditional manual screening requires recruiters to inspect education, skills, experience and certifications individually.

The proposed application should accept a resume, extract meaningful information from it, compare the candidate profile with a job description, and present useful screening information in a structured form. The system should reduce repetitive manual work while keeping the final decision under human control.

### 3. Motivation

The motivation for developing this project comes from the increasing use of digital resumes and the need for faster recruitment workflows. Resume documents may contain the same information in different layouts and formats, making manual comparison time-consuming.

An automated screening assistant provides an opportunity to apply programming and AI/ML concepts to a real-world problem. It also provides a practical foundation for future improvements such as semantic search, automated ranking, dashboards and candidate analytics.

### 4. Objectives

The primary objective is to develop an application that automatically analyzes resumes and provides structured screening information.

- Accept resume documents in common formats such as PDF and DOCX.
- Extract readable text from uploaded resumes.
- Identify candidate contact information where available.
- Extract technical and professional skills using a structured skill dictionary and aliases.
- Identify education, certifications and experience-related information.
- Read and analyze job requirements.
- Compare candidate information with job requirements.
- Generate a candidate matching or screening score.
- Present the results in a simple user interface.
- Provide a modular backend that can be tested and extended.
- Reduce repetitive manual resume-screening effort.

### 5. Scope of the Project

The current project focuses on automated resume analysis and job matching. The system can process structured and semi-structured information found in resumes and provide screening-related results.

The scope includes PDF/DOCX text extraction, text preprocessing, skill identification, contact extraction, education and experience analysis, job-description processing, candidate matching and result presentation.

The project is intended as an academic and assistive screening system. It does not make final hiring decisions, guarantee candidate suitability, or replace human evaluation. Resume formats with scanned images or highly complex layouts may require additional OCR or document-layout processing.

## 6. Existing System

In a traditional recruitment workflow, resumes are collected and manually reviewed by recruiters or hiring teams. Each resume is opened, read and compared with the job description. Relevant skills, education and experience are then recorded or remembered during shortlisting.

This process can become difficult when the number of applicants increases. Manual screening may also produce inconsistent evaluation because different reviewers may focus on different parts of a resume.

Existing keyword-based systems can improve speed, but simple keyword matching may fail when equivalent skills are written using different names. Therefore, a structured parsing and matching system provides a useful foundation for more intelligent screening.

## 7. Proposed System

The proposed AI Resume Screening Studio is a software application that processes resume files and extracts useful candidate information. The backend receives the document, determines its type, extracts text, and passes the content through parsing and analysis modules.

The extracted skills and other candidate attributes are compared with job requirements. The system then presents the identified information and matching results to the user.

The modular design separates document extraction, parsing, matching and API handling. This makes the application easier to maintain and allows future AI/ML components to be added without redesigning the complete system.

## 8. System Requirements

### Hardware Requirements

| Component | Minimum Requirement                                                      |
|-----------|--------------------------------------------------------------------------|
| Processor | Intel Core i3 / AMD equivalent or higher                                 |
| RAM       | 4 GB minimum; 8 GB recommended                                           |
| Storage   | At least 500 MB free for project and dependencies                        |
| Display   | Standard monitor/laptop display                                          |
| Network   | Internet connection recommended for development and package installation |

## Software Requirements

| Software            | Requirement                                                     |
|---------------------|-----------------------------------------------------------------|
| Operating System    | Windows 10/11, macOS, or Linux                                  |
| Python              | Python 3.13 used for the current development environment        |
| Backend             | FastAPI and Uvicorn                                             |
| Database            | SQLAlchemy/PyMySQL components are included for database support |
| Document Processing | PyPDF and python-docx                                           |
| Editor              | Visual Studio Code or equivalent IDE                            |
| Browser             | Modern web browser for API/frontend access                      |

## 9. Technologies and Libraries Used

| Technology / Library | Purpose                                         |
|----------------------|-------------------------------------------------|
| Python               | Primary programming language                    |
| FastAPI              | Backend REST API framework                      |
| Uvicorn              | ASGI server used to run the FastAPI application |
| PyPDF                | PDF text extraction                             |
| python-docx          | DOCX document text extraction                   |
| Pydantic Settings    | Application configuration and settings          |
| SQLAlchemy           | Database abstraction and ORM support            |
| PyMySQL              | MySQL database connectivity                     |
| pytest               | Testing Python modules and application logic    |
| HTTPX                | HTTP client/testing support                     |
| Visual Studio Code   | Development environment                         |

## 10. System Architecture

The system follows a modular processing pipeline. A resume is uploaded through the application interface and sent to the backend. The backend validates the file and selects the appropriate document parser.

The extracted text is passed to the resume parser. The parser identifies skills, contact information, education, experience and certifications. Job-description information is processed separately. The matching component compares candidate attributes with job requirements and produces screening results.

A simplified architecture is shown below:

| Stage                       | Processing                                  |
|-----------------------------|---------------------------------------------|
| 1. User Input               | Resume file and job description             |
| 2. File Validation          | Check supported file type and input         |
| 3. Document Extraction      | Read text from PDF/DOCX                     |
| 4. Text Parsing             | Normalize and identify useful resume fields |
| 5. Skill Extraction         | Detect known skills and aliases             |
| 6. Job Requirement Analysis | Identify required skills and experience     |
| 7. Matching                 | Compare candidate profile with requirements |
| 8. Scoring                  | Calculate screening/matching indicators     |
| 9. API Response             | Return structured screening results         |
| 10. User Interface          | Display candidate analysis                  |

## 11. Input and Dataset Description

The primary input to the system is a candidate resume, normally provided as a PDF or DOCX document. The resume may contain personal contact information, professional summary, skills, education, experience and certifications.

The current parser contains a structured skill dictionary and skill aliases. This allows different written forms of selected technologies or skills to be mapped to a common skill name. Job descriptions provide the second major input and contain the requirements against which the candidate profile is evaluated.

The system is therefore based on document input and structured parsing rules rather than a fixed numerical training dataset. The quality of screening depends on the quality of the parser rules, skill vocabulary and job-description information.

## 12. Resume Text Extraction and Preprocessing

Resume documents can contain different formatting, capitalization, punctuation and layouts. Preprocessing is therefore required before extracting structured information.

1. Identify the uploaded file type.
2. Extract text from PDF using PyPDF.
3. Extract paragraphs and text from DOCX using python-docx.
4. Normalize whitespace and unnecessary formatting.
5. Search normalized text using structured patterns and skill aliases.
6. Preserve important technical terms while extracting candidate information.

The parser is designed to provide a consistent internal representation of resume information so that later matching operations can work with structured fields instead of raw document formatting.

## 13. Skill Extraction

Skill extraction is one of the main functions of the system. The parser uses a predefined skill vocabulary and aliases to recognize technical and professional skills appearing in resume text.

For example, different textual forms of a technology can be mapped to a standard skill name. This reduces the effect of minor differences in resume wording and makes candidate-job comparison more consistent.

| Input Information   | Structured Output      |
|---------------------|------------------------|
| Resume text         | Full extracted text    |
| Technical keywords  | Normalized skills      |
| Skill aliases       | Standard skill names   |
| Education terms     | Education information  |
| Experience terms    | Experience information |
| Certification terms | Certification list     |

## 14. Job Description Matching

The job description is analyzed to identify the skills and requirements expected for a position. The candidate's extracted skills are then compared against these requirements.

A simple matching process can identify matched skills and missing skills. A more advanced implementation can assign different weights to required and preferred skills.

| Matching Result  | Meaning                                                       |
|------------------|---------------------------------------------------------------|
| Matched Skills   | Skills present in both candidate profile and job requirements |
| Missing Skills   | Important job skills not identified in the resume             |
| Candidate Skills | Skills extracted from the resume                              |
| Job Requirements | Skills and requirements extracted from the job description    |
| Match Score      | Overall indicator of alignment between candidate and position |



## 15. Candidate Scoring and Ranking

Candidate scoring provides a compact way to communicate how closely a resume aligns with the supplied job requirements. A basic score can be calculated from the proportion of required skills identified in the candidate profile.

For example, if a job description contains a set of required skills, the system can calculate the percentage of those requirements matched by the resume. Additional weighting can be introduced for experience, education or mandatory requirements.

The score should be treated as a screening indicator rather than a final hiring decision. Human review remains important because resumes may contain information that automated rules cannot fully understand.

## 16. Screening Algorithm

7. Start the application.
8. Load application configuration and required modules.
9. Receive a resume file from the user.
10. Validate the file format.
11. Extract text from the PDF or DOCX document.
12. Normalize the extracted text.
13. Extract contact information where available.
14. Extract skills using the skill vocabulary and aliases.
15. Extract education, experience and certifications.
16. Receive or process the job description.
17. Extract job-related skills and requirements.
18. Compare candidate skills with job requirements.
19. Calculate the screening/matching score.
20. Prepare matched and missing skill information.
21. Return structured results to the user interface.
22. Display the screening report.
23. End or process another resume.

## 17. Detailed Module Description

| Module              | Purpose                                                         |
|---------------------|-----------------------------------------------------------------|
| backend/app/main.py | Application entry point and FastAPI route handling              |
| parser.py           | Resume document extraction and structured resume parsing        |
| PDF Processing      | Reads text from PDF resume documents                            |
| DOCX Processing     | Reads text from Word documents                                  |
| Skill Extraction    | Detects and normalizes known skills and aliases                 |
| Job Matching        | Compares candidate skills with job requirements                 |
| Configuration       | Stores application settings and environment configuration       |
| Database Layer      | Provides database support through SQLAlchemy/PyMySQL components |
| Testing             | Validates application and parser functionality                  |

## 18. User Interface and Interaction

The application is designed to provide a simple workflow for resume screening. The user provides a resume and job-related requirements, after which the backend processes the document and returns structured information.

The result interface can present candidate details, extracted skills, education, experience, certifications, matched skills, missing skills and an overall screening indicator. A clear result presentation helps users understand why a candidate was identified as a potential match.

The interface should be considered an assistive tool. Users should review the original resume before making important recruitment decisions.

## 19. Installation and Execution

The current development environment uses Python 3.13 and a project-specific virtual environment.

24. Open the project folder in Visual Studio Code.
25. Create the virtual environment using: `py -3.13 -m venv .venv`
26. Activate it using: `.\venv\Scripts\Activate.ps1`
27. Install dependencies using: `python -m pip install -r requirements.txt`
28. Run the FastAPI backend using: `python -m uvicorn backend.app.main:app`
29. For development with automatic reload, use: `python -m uvicorn backend.app.main:app --reload`
30. Open the FastAPI documentation page at <http://127.0.0.1:8000/docs> when the server is running.

The exact frontend command depends on the frontend folder and its package configuration. If a frontend containing package.json is included, its dependencies should be installed with `npm install` and started using the script defined in package.json.

## 20. Testing Strategy

Testing should verify document extraction, parsing, API behavior and matching logic. Different resume formats and different job descriptions should be tested to identify parsing and matching problems.

| Test Case                   | Expected Result                                         |
|-----------------------------|---------------------------------------------------------|
| Valid PDF resume            | Text extracted successfully                             |
| Valid DOCX resume           | Text extracted successfully                             |
| Unsupported file            | Validation error returned                               |
| Resume with skills          | Relevant skills extracted                               |
| Resume with education       | Education information identified                        |
| Resume with experience      | Experience information identified                       |
| Job description with skills | Required skills identified                              |
| Matching skills             | Matched skills returned                                 |
| Missing skills              | Missing requirements identified                         |
| Empty/invalid input         | Clear validation response                               |
| API startup                 | FastAPI application starts without syntax/import errors |

## 21. Results and Discussion

The implemented project provides a backend foundation for automated resume screening. The application can load supported resume documents, extract text, and use the parser to identify structured candidate information.

The use of a predefined skill vocabulary and aliases helps normalize common technical terms before matching. This provides a more useful result than comparing raw resume text alone.

The current system is particularly suitable for demonstrating automated resume parsing and rule-based candidate-job matching. More advanced semantic methods can improve the handling of synonyms, context and differently worded requirements.

## 22. Advantages

- Reduces repetitive manual resume screening work.
- Supports common PDF and DOCX resume formats.
- Extracts useful candidate information into structured fields.
- Provides normalized skill recognition using aliases.
- Allows candidate requirements to be compared systematically.
- Can be extended with machine-learning and NLP techniques.
- FastAPI provides a clear and scalable backend structure.
- Modular code makes testing and maintenance easier.
- Can support large improvements in screening workflow when integrated with a suitable frontend.

## 23. Limitations

- Parsing quality depends on the structure and readability of the resume.
- Scanned image-only resumes may require OCR support.
- Rule-based skill extraction cannot understand every possible synonym or context.
- Job descriptions may contain ambiguous requirements that are difficult to interpret automatically.
- Matching scores are indicators and should not be treated as final hiring decisions.
- Unusual resume layouts may require additional document-layout processing.
- The quality of results depends on the completeness of the skill vocabulary and parsing rules.

## 24. Applications

- Recruitment and HR resume pre-screening.
- College placement and internship screening.
- Candidate-job matching systems.
- Career portals and recruitment platforms.
- Internal employee skill-profile systems.
- Resume analysis and career-assistance applications.
- Academic demonstrations of NLP, AI and document processing.

## 25. Future Enhancements

- Add semantic embeddings for meaning-based resume-job matching.
- Add OCR for scanned resumes.
- Use machine-learning models for candidate classification.
- Improve experience and education extraction using NLP.
- Add weighted scoring for mandatory and preferred requirements.
- Provide candidate ranking and shortlist dashboards.
- Add recruiter authentication and role-based access.
- Store screening history in a database.
- Add analytics and screening statistics.
- Support more resume formats.
- Add explainable scoring showing why a candidate received a particular score.
- Integrate advanced AI models for contextual skill and requirement understanding.

## 26. Conclusion

AI Resume Screening Studio demonstrates how Python, document processing and AI/ML-oriented text analysis can be combined to build a practical recruitment-support application. The system processes resume documents, extracts meaningful candidate information and compares that information with job requirements.

The project provides a structured foundation for automated screening while keeping the final evaluation under human control. Its modular architecture allows future improvements such as semantic embeddings, machine learning, OCR, candidate ranking and analytics.

Overall, the project shows that automated resume analysis can reduce repetitive screening work and provide recruiters or academic placement teams with organized information for further candidate evaluation.

## 27. References

31. Python Software Foundation – Python Documentation.
32. FastAPI Documentation – FastAPI framework and API development.
33. Uvicorn Documentation – ASGI server documentation.
34. PyPDF Documentation – PDF text extraction.
35. python-docx Documentation – Microsoft Word document processing.
36. SQLAlchemy Documentation – Python SQL toolkit and ORM.
37. Pydantic Documentation – Data validation and settings management.
38. pytest Documentation – Python testing framework.
39. scikit-learn Documentation – Machine learning and text-processing concepts.
40. Natural Language Processing and Information Retrieval references for resume text analysis.

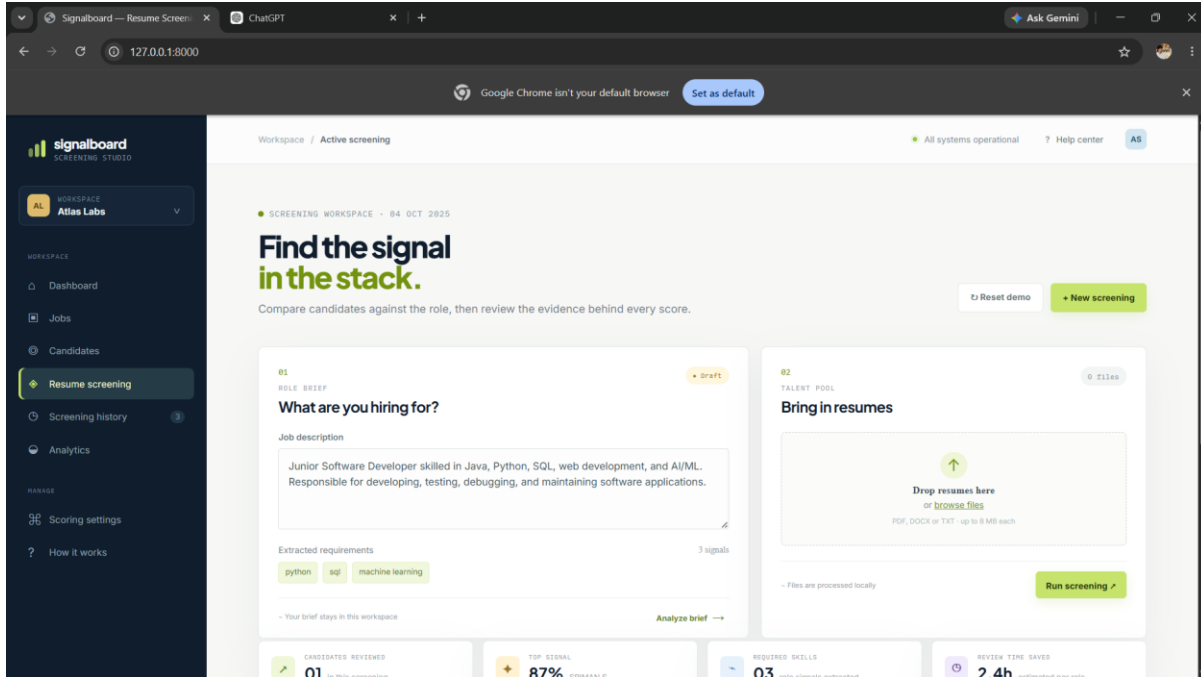
## 28. Project File Structure

| File / Folder           | Purpose                                     |
|-------------------------|---------------------------------------------|
| backend/app/main.py     | FastAPI application entry point             |
| backend/app/parser.py   | Resume extraction and parsing logic         |
| backend/app/            | Backend application modules                 |
| requirements.txt        | Python project dependencies                 |
| .venv/                  | Project-specific Python virtual environment |
| Resume files (PDF/DOCX) | Candidate input documents                   |
| Frontend folder         | User interface, if included in the project  |
| Database configuration  | Database connection and persistence support |

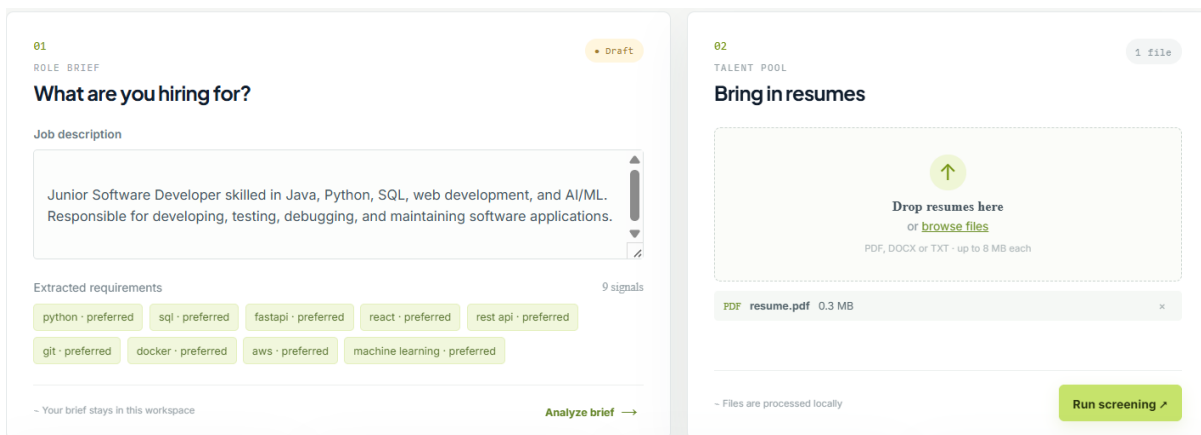
Note: The file structure above describes the project components currently established for the AI Resume Screening Studio. Additional frontend or database files may vary depending on the final project implementation.

## OUTPUT :

Open Webpage:



Resume Scanning:



## After Scanned A Resume:

01

ROLE BRIEF

Draft

### What are you hiring for?

Job description

Junior Software Developer skilled in Java, Python, SQL, web development, and AI/ML. Responsible for developing, testing, debugging, and maintaining software applications.

Extracted requirements

python sql machine learning

3 signals

~ Your brief stays in this workspace

Analyze brief →

02

TALENT POOL

0 files

### Bring in resumes

↑

Drop resumes here  
or [browse files](#)

PDF, DOCX or TXT - up to 8 MB each

~ Files are processed locally

Run screening →

CANDIDATES REVIEWED

01

in this screening

TOP SIGNAL

87%

SRIMAN S

REQUIRED SKILLS

03

role signals extracted

REVIEW TIME SAVED

2.4h

estimated per role

## Eligibility Of Candidate Got:

SCREENING OUTPUT

Candidate signals

1 of 1 candidates - ranked by signal strength

Search candidates

All signals

All skills

Min 0 %

Score: high to low

| #  | CANDIDATE                    | SIGNAL | SKILLS    | EXPERIENCE | EDUCATION |
|----|------------------------------|--------|-----------|------------|-----------|
| 01 | SS<br>SRIMAN S<br>resume.pdf | 87%    | 9 matched | 100%       | 100%      |

Signalboard baseline - transparent scoring enabled

Download report ↓

ROLE PULSE

LIVE

### The strongest pattern

SRIMAN S leads with aws, git, javascript in the role signal.

Required skills

86%

Experience

73%

Text alignment

68%

Keep humans in the loop  
Review the evidence before taking action. A missing keyword is not a missing capability.

## Candidate Data Storing :

Signalboard — Resume Screen

127.0.0.1:8000

Google Chrome isn't your default browser

Set as default

signalboard

SCREENING STUDIO

WORKSPACE

AL

Atlas Labs

WORKSPACE

Dashboard

Jobs

Candidates

Resume screening

Screening history

Analytics

MANAGE

Scoring settings

How it works

Responsible by default

Support support review. They never decide it.

AS

Alex Singh

Responsible lead

Workspace / Active screening

All systems operational

Help center

AS

TALENT POOL

## People in your review set

A focused view of candidates currently attached to this screening.

Back to screening

SS

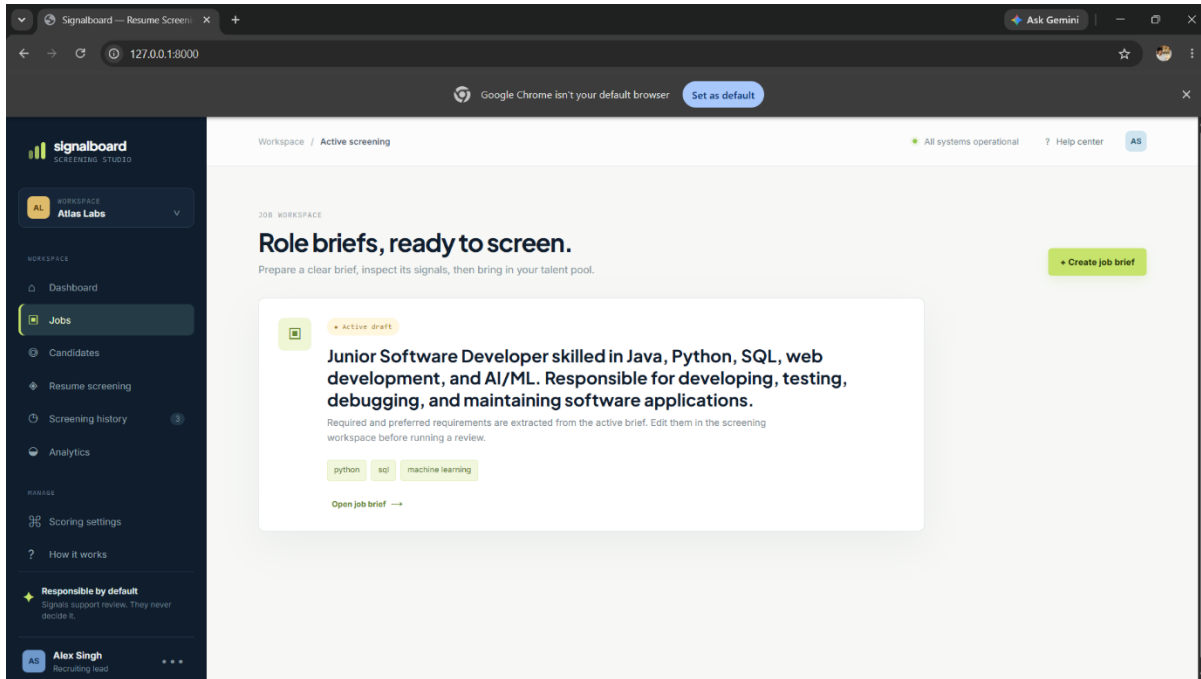
SRIMAN S

srman50703004@gmail.com

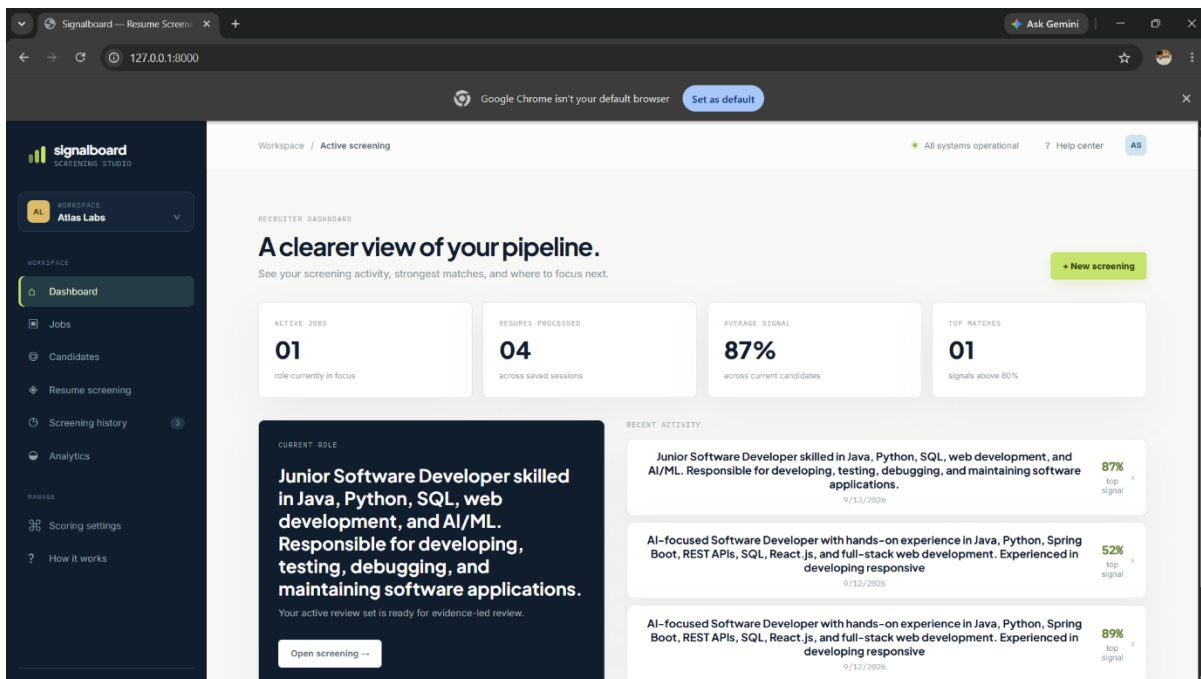
9 matching skills

87%

## Jobs Page:

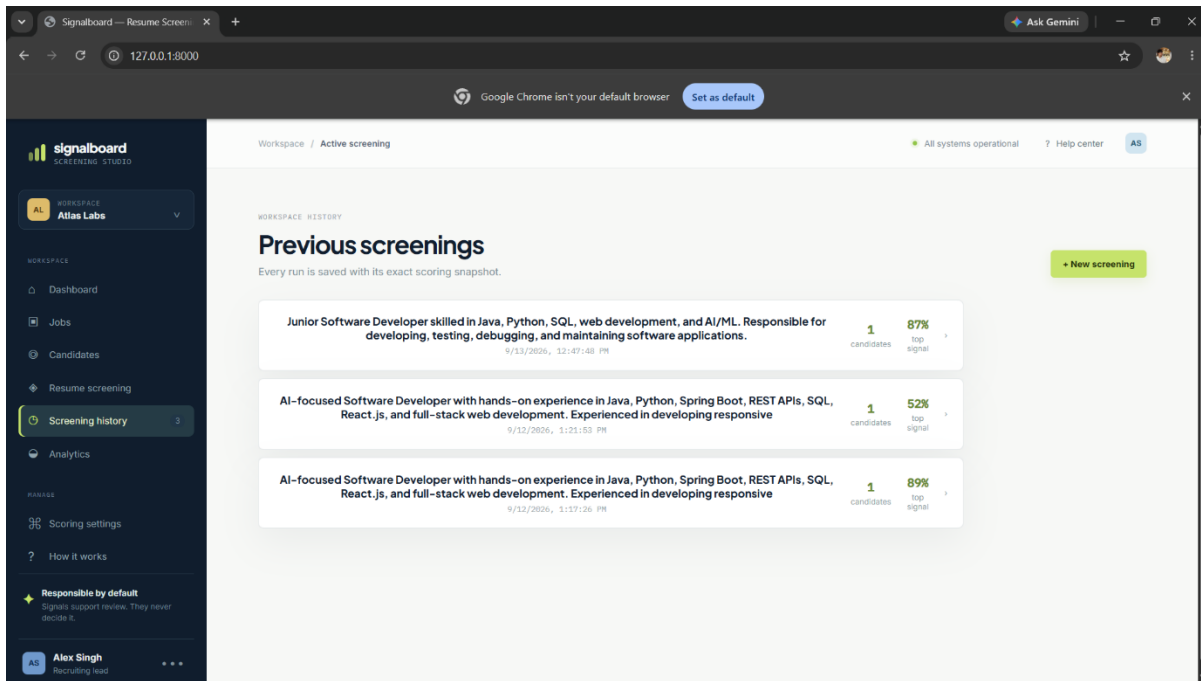


## Dashboard Page:

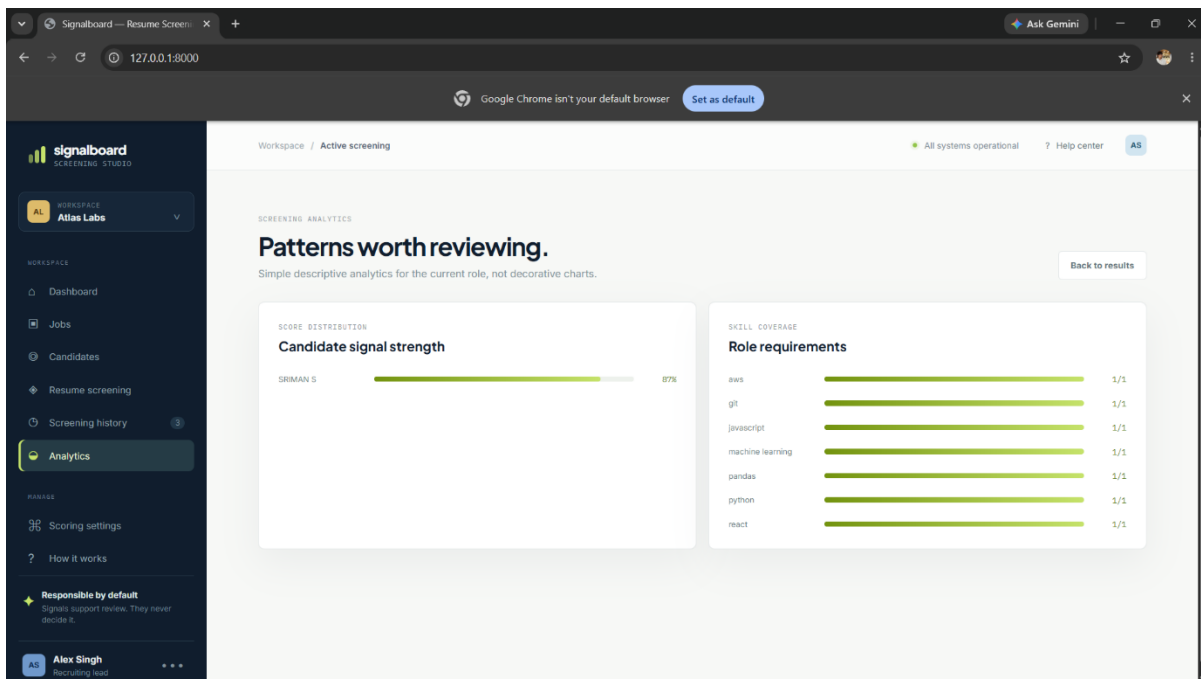




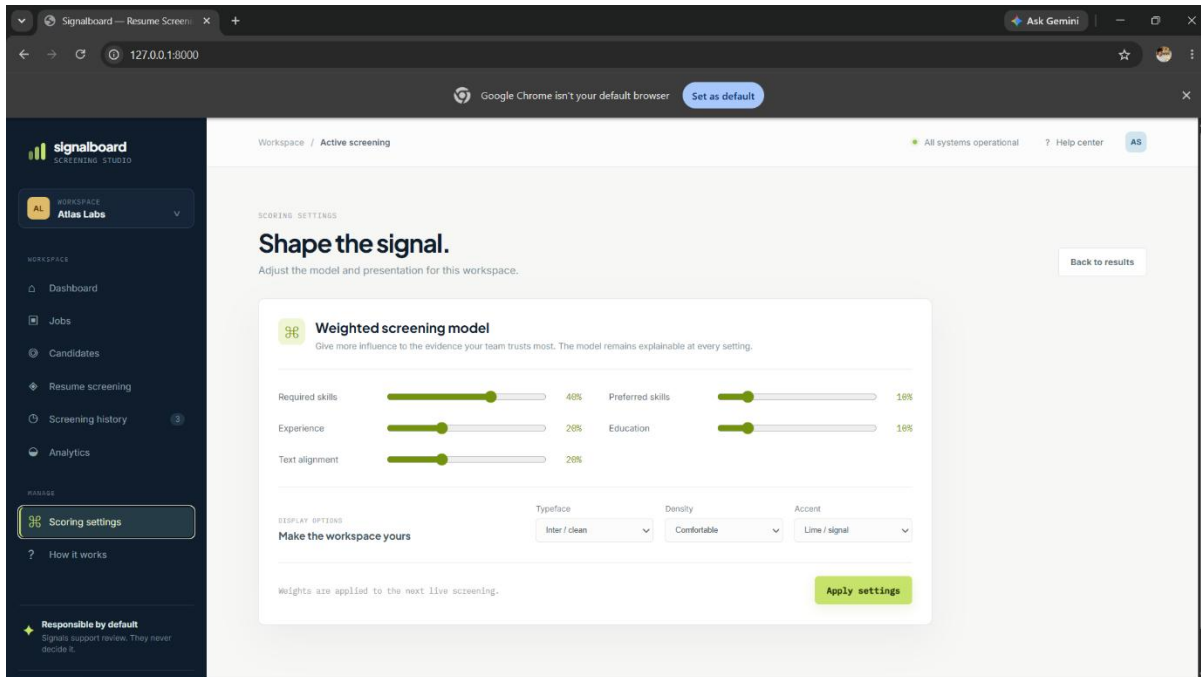
## Screening History:



## Analytics Page:



## Customize Scanning Settings:



# THANK YOU